UNIVERSIDADE ESTADUAL DE MARINGÁ
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

ALGORITMOS GENÉTICOS: PROBLEMA DO CAIXEIRO VIAJANTE

Discente: Vinícius Schmitz Freitas de Oliveira **RA:** 109887

Docente: Ademir Constantino

Disciplina: Modelagem e Otimização Algorítmica

Maringá, 03 de fevereiro de 2025

SUMÁRIO

1. Sumário
2. Introdução
3. Análise do Algoritmo
4. Testes e Resultados

INTRODUÇÃO

Este relatório descreve a implementação de um algoritmo genético combinado com busca local para resolver o Problema do Caixeiro Viajante (TSP). O objetivo foi desenvolver e avaliar diferentes estratégias de cruzamento e mutação, além de procurar por calibrar os parâmetros para obter soluções de melhor qualidade na tentativa de evitar a convergência prematura que ocorre quando o TSP a solução converge para um valor sub-ótimo muito rapidamente, perdendo a diversidade da população e limitando a exploração de outras possíveis soluções melhores.
O algoritmo foi implementado em C e utiliza uma abordagem baseada em algoritmos genéticos, com funções de **avaliação** com base no fitness de cada indivíduo, de **seleção** por classificação, de **cruzamento** utilizando operador OX1 e OX2, e por fim **mutação** para explorar o espaço de soluções.

ESTRUTURA DO PROGRAMA

- **<u>VARIÁVEIS E DEFINIÇÕES GLOBAIS</u>:**
  1. **Definições**
     - ☑ ***TAM_POPULACAO***, tamanho da população
     - ☑ ***TAM_SOLUCAO***, tamanho da solução (deve ser correspondente ao arquivo.tsp)
     - ☑ ***SOLUCAO_IDEAL***, melhor solução conhecida da literatura para o problema em questão, utilizado para calcular o *fitness*
  2. **Variáveis**
     - ☑ ***int **populacaoAtual***; população de soluções/indivíduos
     - ☑ ***classificacao[TAM_POPULACAO];*** classificação realizada na fase de <u>seleção</u>

- ☑ **unsigned long long int distTotal[TAM_POPULACAO]**; distâncias totais de cada solução/indivíduo
- ☑ ***fitness***; fitness de cada solução/indivíduo
- ☑ **novasDists[2]**; distâncias totais dos 2 filhos gerados no crossover
- ☑ **typedef struct {**
  float x, y;
  int visitado;
  } Cidades; estrutura para guardar as cidades do arquivo
- ☑ **Cidades cidades[TAM_SOLUCAO]**; vetor contendo as cidades

❖ **<u>FUNÇÕES</u>:**

1. **Do código anterior**
   - ☑ **void limparVisitados(void);**
   - ☑ **void mostrarSolucao(int *);**
   - ☑ **int calculaDistancia(Cidades, Cidades);**
   - ☑ **void lerArquivo(char *);**
   - ☑ **void buscaLocalVizinhoMaisProximo(int *, int);**

2. **Funções auxiliares novas**
   - ☑ **void mostrarPopulacao(void);**
   - ☑ **void cortarSegmento(int*, int*);**
   - ☑ **int verificaInseridoFilho(int*, int, int);**
   - ☑ **unsigned long long int calculaDistTotal(int*);**
   - ☑ **long int iniciarPairswap(int*);**
   - ☑ **void copiarFilho(int*, int*);**
   - ☑ **void gerarIndividuoAleatorio(int*);**
   - ☑ **void aleatoriezarPopulacao(void);**

3. **Funções novas principais e hierarquia entre elas**
   - ☑ **void gerarNovaSolucao(int *, int*, int);**
   - ☑ **void alocarMemoria(void);**
   - ☑ **void inicializarPopulacao(void);**
     - ☑ **void buscaLocalVizinhoMaisProximo(int*, int);**

- ☑ **int rodarAlgoritmoGenetico(void);**
    - ☑ **void iniciarAvaliacao(void);**
    - ☑ **void iniciarSelecao(void);**
    - ☑ **void iniciarCruzamentoOX1(int*, int*);**
    - ☑ **void iniciarCruzamentoOX2(int*, int*);**
    - ☑ **void iniciarMutacao(int *, int*);**
    - ☑ **int atualizarPopulacao(int*, int*);**

❖ **<u>FASES DE IMPLEMENTAÇÃO</u>:**

1. **Leitura dos dados de entrada (trabalho anterior)**
    - ☑ leitura do arquivo.tsp:
        - realizada na função **lerArquivo()**

2. **Armazenamento dos dados de entrada (trabalho anterior)**
    - ☑ Cidades: inicializado globalmente, dados são armazenados em um ***array* de *structs*** representando os nós/cidades (dados de entrada do arquivo), contendo:
        - int x,y, sendo as coordenadas do nó
        - int visitado = 0, sendo para auxiliar no algoritmo de busca na vizinhança

3. **Estruturas de armazenamento de uma solução**
    - ☑ Individuo/Solução: inicializados globalmente, espaço alocado dinamicamente na função **alocarMemoria()** com malloc(), ***array* global de inteiros** representando a sequência de cidades visitadas
    - ☑ Filhos: alocados na **main()** como **2 *arrays* de inteiros**, representam os filhos que serão gerados a partir de 2 pais com um operador de cruzamento

4. **Estruturas de armazenamento da população de soluções**

- ☑ População: inicializada globalmente, espaço alocado dinamicamente na função **alocarMemoria()**
    - **populacaoAtual ponteiro de ponteiros, atua como uma matriz onde cada linha é uma solução/indivíduo e as colunas são os nós
- ☑ População inicial: gerada na função **inicializarPopulação()** com a heurística do vizinho mais próximo para cada indivíduo, para gerar variação é alterado o nó inicial gerado randomicamente a cada chamada

5. **Algoritmo Genético**
    a. **Avaliação**
        - Fitness: diferença para a melhor solução
    b. **Seleção***
        - Por Classificação: vetor com melhor fitness na posição 0 e decresce Fitness até a última posição

    c. **Cruzamento***
        - OX1
        - OX2
    d. **Mutação***
        - Pair-swap: aproveitada a função do trabalho anterior, troca um par de cidades para cada filho gerado no operador de cruzamento
        - Aleatorização: função auxiliar para tentar impedir a ocorrência de convergência prematura, aleatorizando ⅓ do grupo de soluções antes do cruzamento
    e. **Manutenção da população***
        - Elitismo

ANÁLISE DO ALGORITMO

INFLUÊNCIA DOS PARÂMETROS NAS SOLUÇÕES

Alguns parâmetros influenciavam muito no espaço de soluções que era obtido nos testes, como a quantidade de grupos do algoritmo que quanto mais grupos mais demorava para o algoritmo convergir prematuramente, porém com muitos grupos o algoritmo não conseguia avançar tão bem para encontrar soluções melhores, **a melhor quantidade de grupos foi 50 de acordo com os testes**.
Tal como a escolha dos indivíduos pais para os operadores de cruzamento, onde inicialmente era escolhido os 2 melhores de acordo com o Fitness, mas foi alterado para a escolha de 2 aleatórios, **testes mostraram que a melhor opção seria considerar os 2 métodos escolhendo o melhor pai + um pai aleatório**. Assim, também foi adicionada uma etapa no algoritmo que é a de aleatorização da população onde ⅓ dessa era aleatorizada a cada X quantidade de iterações (um parâmetro ajustável, **foi escolhido a cada 50 iterações como o valor base**.

NOVAS FUNÇÕES

Order Crossover (OX1) - reprodução

void iniciarCruzamentoOX1(int *filho1, int *filho2)

Implementa o operador de cruzamento do tipo OX1 (Order Crossover 1) para gerar duas novas soluções a partir da seleção de 2 soluções pais, aqui o principal ajuste feito foi em relação aos pais escolhidos, anteriormente foi definido como critério de seleção as 2 melhores soluções (calculadas pela função de seleção, no vetor classificação) porém assim os filhos convergiam prematuramente para um espaço muito restrito de soluções, então foram feitos testes e ficou decidido que a melhor opção seria a escolha de um pai como o com o melhor Fitness, e outro aleatório da população de soluções

Funcionamento:

1. Escolhe o melhor indivíduo da população (`pai1`) e outro aleatório (`pai2`).
2. Define dois pontos de corte aleatórios no cromossomo.
3. Copia o segmento entre esses pontos do `pai1` para o `filho1` e do `pai2` para o `filho2`.
4. Preenche as posições restantes dos filhos com os genes do outro pai, garantindo que não haja repetições.
5. Calcula as novas distâncias totais (`novasDists[0]` e `novasDists[1]`).

Observação:

- Mantém a ordem original dos genes fora do segmento copiado.
- Função cortarSegmento() escolhe 2 pontos de corte dos pais.
- O uso da função `verificaInseridoFilho()` evita duplicação de elementos.

Order Based Crossover 2 (OX2) - reprodução

iniciarCruzamentoOX2(int *filho1, int *filho2)

Implementa o operador de cruzamento OX2, que difere do OX1 ao selecionar aleatoriamente genes dos pais para os filhos. Aqui para diferenciar mais do outro operador, manteve-se a escolha dos 2 pais de acordo com os 2 melhores fitness pois é feita a tentativa de gerar mais aleatoriedade escolhendo posições aleatórias para troca.

Funcionamento:

1. Escolhe dois pais diferentes na população.
2. Seleciona aleatoriamente genes do `pai1` e do `pai2` e os insere nos filhos.
3. Preenche as posições restantes mantendo a ordem dos genes no outro pai.
4. Calcula a nova distância total para cada filho.

Observação:

- Esse método busca preservar mais diversidade genética ao não fixar um segmento específico do pai.

Mutação - pair swap

iniciarMutacao(int *filho1, int *filho2)

Aplica mutação utilizando a técnica de **pair-swap** para melhorar a solução, anteriormente a função foi retirada do trabalho anterior onde ela encerrava ao encontrar uma solução melhor. Foi alterado que a função apenas troque um par e

retorne a diferença no custo total, independente se a solução melhorou ou não. Isso foi feito para que se explorasse um espaço maior de soluções a fim de evitar a convergência prematura.

Funcionamento:

1. Chama `iniciarPairswap()` para trocar um par para `filho1` e `filho2` separadamente.
2. Devolve a diferença de custo total com a troca de par

Observação:

- A mutação tenta trocar duas cidades na rota, buscando explorar um maior espaço de soluções, não necessariamente sendo uma mais eficiente.

Elitismo e Aleatório - manutenção

`atualizarPopulacao(int *filho1, int *filho2)`

Descrição:
Substitui os piores indivíduos da população se os filhos gerados forem melhores.

Funcionamento:

1. Obtém os dois piores indivíduos (`pior` e `segundoPior`) com base na classificação.
2. Se `filho1` for melhor que o pior, ele o substitui.
3. Se `filho2` for melhor que o segundo pior, ele o substitui.
4. Retorna `1` se houve atualização na população, `0` caso contrário.

Observação:

- Esse método ajuda a manter a evolução das soluções, garantindo que a população sempre melhore.

Aleatorização - mutação 2

aleatoriezarPopulacao()

Descrição:
Renova parte da população, introduzindo diversidade.

Funcionamento:

1. Escolhe aproximadamente **1/3** da população intermediária.
2. Gera novos indivíduos aleatórios para substituir essas soluções.
3. Recalcula a distância total dessas novas soluções.

Observação:

- Essa estratégia evita **convergência prematura** para um ótimo local ao introduzir variabilidade.

CONCLUSÃO GERAL E QUESTÕES PARA OTIMIZAÇÃO

- **Cruzamento (OX1 e OX2)** garantem a recombinação eficiente dos genes.
    - Quais pais selecionar?
- **Mutação (pair-swap)** busca inserir uma pequena mutação dentro dos filhos gerados.
- **Atualização da população** assegura que sempre sejam mantidas melhores soluções e piores sejam descartadas.
    - Quais indivíduos substituir na manutenção?

- **Aleatorização** impede que o algoritmo fique preso em ótimos locais sem explorar soluções melhores.
    - Quais e quantos indivíduos da população aleatorizar?
    - A cada quantas iterações aleatorizar?
    - Onde no fluxo do algoritmo genético inserir a aleatorização?
- **Quantidade de grupos** foram feitos inicialmente testes com quantidades de 5, 10, 20, 50, 100 e 500 grupos. E foi constatado que a melhor quantidade foi de 50 grupos, para as instâncias muito grandes (pla) por conta da demora para inicialização da população, foram utilizados apenas testes com 5 grupos

TESTES E RESULTADOS

ur574

Decrescente - solução

Solução	Tempo de Processamento	Grupos	Iterações	Operador
42.357	3404s	500	5 X 10^6	OX2
42.911	**457s**	100	5 x 10^6	OX2
42.946	10.115s	20	5 x 10^6	OX1
43.272	4333s	500	5 X 10^6	OX1
43.283	1003s	50	5 x 10^6	OX1
43.663	315s	50	5 x 10^6	OX2
43.945	272s	20	5 x 10^6	OX2

Solução	Tempo de Processamento	Grupos	Iterações	Operador
44.448	67s	50	1 x 10^6	OX2
44.657	887s	500	1 x 10^6	OX1

Diferença da melhor solução encontrada para solução obtida no trabalho anterior:
46.924 (-3.641)

pcb1173

Decrescente - solução

Solução	Tempo de Processamento	Grupos	Iterações	Operador
68.170	**457s**	50	5 x 10^6	OX1
69.471	3298s	100	5 x 10^6	OX1
69.677	579s	100	5 x 10^6	OX1
69.766	879s	100	5 x 10^6	OX2
70.191	963s	50	5 x 10^6	OX2
70.491	183s	100	1 x 10^6	OX2

Diferença da melhor solução encontrada para solução obtida no trabalho anterior:
71.010 (-2.840)

pr1002

Decrescente - solução

Solução	Tempo de Processamento	Grupos	Iterações	Operador
307.295	2215s	50	5 x 10^6	OX1
311.120	**633s**	50	5 x 10^6	OX2

Diferença da melhor solução encontrada para solução obtida no trabalho anterior:
315.346 (-8.051)

brd14051

Decrescente - solução

Solução	Tempo de Processamento	Grupos	Iterações	Operador
565058	**7545s**	50	5 x 10^6	OX1
567.025	9132s	50	5 x 10^6	OX2

Diferença da melhor solução encontrada para solução obtida no trabalho anterior:
575.083 (-10.025)

fnl4461

Decrescente - solução

Solução	Tempo de Processamento	Grupos	Iterações	Operador
220.413	6430s	50	1 x 10^6	OX1

Solução	Tempo de Processamento	Grupos	Iterações	Operador
221.933	**2838s**	50	5 x 10^6	OX2

Diferença da melhor solução encontrada para solução obtida no trabalho anterior:
223.855 (-3.442)

d15112

Decrescente - solução

Solução	Tempo de Processamento	Grupos	Iterações	Operador
1.915.832	**1s**	10	500	OX2
1.920.079	47s	10	500	OX1

Diferença da melhor solução encontrada para solução obtida no trabalho anterior:
1.942.334 (-26.502)

pla33810

Decrescente - solução

Solução	Tempo de Processamento	Grupos	Iterações	Operador
2.181.762.282	**1s**	5	100	OX2
10.771.650.134	71s	5	100	OX1

Diferença da melhor solução encontrada para solução obtida no trabalho anterior:
4.329.349.129 (-2.147.586.850)

pla85900

Decrescente - solução

Solução	Tempo de Processamento	Grupos	Iterações	Operador
4.342.644.399	150s	5	100	OX2
4.342.691.842	**11s**	5	100	OX1

Diferença da melhor solução encontrada para solução obtida no trabalho anterior:
4.342.697.983 (-53.584)